State Circuit (상태 회로)

개요

State Circuit은 ZKEVM에서 EVM 상태 변경의 정확성을 검증하는 핵심 회로입니다. 이 회로는 Read-Write (RW) 테이블의 모든 연산이 올바른 순서로 정렬되어 있고, 각 상태 변경이 Merkle Patricia Trie (MPT)와 일치하는지 검증합니다.

주요 역할:

- RW 테이블의 lexicographic ordering (사전식 정렬) 검증
- 상태 읽기/쓰기 연산의 일관성 검증
- MPT 업데이트와 상태 루트 변경 검증
- 각 RW 연산 타입별 제약 조건 적용

파일 위치:

- `zkevm-circuits/src/state_circuit.rs` - 메인 회로 구현
- `zkevm-circuits/src/state_circuit/constraint_builder.rs` - 제약 조건 빌더
- `zkevm-circuits/src/state_circuit/lexicographic_ordering.rs` - 정렬 검증
- `zkevm-circuits/src/state_circuit/multiple_precision_integer.rs` - MPI 칩
- `zkevm-circuits/src/state_circuit/random_linear_combination.rs` - RLC 칩
- `zkevm-circuits/src/state_circuit/lookups.rs` - 룩업 테이블

RW Table 구조

RwRow 정의

RW 테이블의 각 행은 하나의 읽기 또는 쓰기 연산을 나타냅니다:

```
pub struct RwTable {
pub q_enable: Column<Fixed>, // 활성화 플래그
pub rw_counter: Column<Advice>, // RW 카운터
pub is_write: Column<Advice>, // 쓰기 여부 (1=쓰기, 0=읽기)
pub tag: Column<Advice>, // 연산 타입 (RwTableTag)
pub id: Column<Advice>, // 트랜잭션/호출 ID
pub address: Column<Advice>, // 주소 (계정 주소, 메모리 주소 등)
pub field_tag: Column<Advice>, // 필드 태그 (Account의 경우)
pub storage_key: Column<Advice>, // 스토리지 키 (256비트)
pub value: Column<Advice>, // 새로운 값
pub value_prev: Column<Advice>, // 이전 값
pub aux1: Column<Advice>, // 보조 데이터 1
pub aux2: Column<Advice>, // 보조 데이터 2 (committed value)
}
```

RwTableTag 열거형

RW 테이블의 각 행은 다음 중 하나의 타입을 가집니다:

```
pub enum RwTableTag {
Start = 1, // 패딩용 시작 행
Stack, // 스택 연산
Memory, // 메모리 연산
```

```
TxAccessListAccount, // 트랜잭션 액세스 리스트 (계정)

TxAccessListAccountStorage, // 트랜잭션 액세스 리스트 (스토리지)

TxRefund, // 트랜잭션 환불

Account, // 계정 상태

AccountStorage, // 계정 스토리지

AccountTransientStorage, // 임시 스토리지 (EIP-1153)

CallContext, // 호출 컨텍스트

TxLog, // 트랜잭션 로그

TxReceipt, // 트랜잭션 영수증

}
```

Reversible 연산: `TxAccessListAccount`, `TxAccessListAccountStorage`, `TxRefund`, `Account`, `AccountStorage`, `AccountTransientStorage` 는 revert 시 되돌릴 수 있는 연산입니다.

정렬 키 시스템 (Sort Keys)

State Circuit의 핵심은 모든 RW 연산이 다음 키들의 조합으로 사전식 정렬되어 있음을 검증하는 것입니다:

정렬 순서

1. **tag** (4비트) - RW 연산 타입
2. **id** (32비트, 2 limbs) - 트랜잭션 또는 호출 ID
3. **address** (160비트, 10 limbs) - 계정 주소 또는 메모리 주소
4. **field_tag** (8비트) - Account 필드 타입

5. **storage_key** (256비트, 32 bytes) - 스토리지 키

6. **rw_counter** (32비트, 2 limbs) - RW 카운터

```
pub struct SortKeysConfig {
tag: BinaryNumberConfig<RwTableTag, 4>,
id: MpiConfig<u32, N_LIMBS_ID>, // 2 limbs
address: MpiConfig<Address, N_LIMBS_ACCOUNT_ADDRESS>, // 10 limbs
field_tag: Column<Advice>,
storage_key: RlcConfig<N_BYTES_WORD>, // 32 bytes
rw_counter: MpiConfig<u32, N_LIMBS_RW_COUNTER>, // 2 limbs
}
```

Lexicographic Ordering 검증

정렬 검증은 512비트 값을 32개의 16비트 limb로 분해하여 수행됩니다:

Limb 인덱스 (Big Endian 순서):

```
pub enum LimbIndex {
Tag, // 0: tag (4비트) + padding
Id1, Id0, // 1-2: id (32비트)
Address9, ..., Address0, // 3-12: address (160비트)
FieldTag, // 13: field_tag (8비트) + padding
StorageKey15, ..., StorageKey0, // 14-29: storage_key (256비트)
RwCounter1, RwCounter0, // 30-31: rw_counter (32비트)
```

```
}
```

검증 알고리즘:

현재 행 X_cur와 이전 행 X_prev를 비교할 때:

- A0, A1, ..., A31: X_cur의 32개 limb (big endian)
- B0, B1, ..., B31: X_prev의 32개 limb (big endian)

차이 계산:

```
C0 = A0 - B0

C1 = C0 << 16 + A1 - B1

...

C31 = C30 << 16 + A31 - B31
```

X_cur > X_prev ⟺ C0, ..., C31 중 하나가 0이 아니고 16비트에 맞음

구현 컬럼:

- `first_different_limb` : 처음으로 다른 limb의 인덱스 (BinaryNumberChip 사용)
- `limb_difference` : 해당 limb의 차이값
- `limb_difference_inverse` : 차이값의 역원 (0이 아님을 증명)

제약 조건:

1. `limb_difference` 가 16비트에 맞음 (u16 룩업)
2. `limb_difference` 가 0이 아님 (역원 존재)
3. `first_different_limb` 이전의 모든 limb 차이의 RLC가 0
4. `limb_difference` 가 해당 인덱스의 limb 차이와 같음

서브 칩 (Sub-Chips)

State Circuit은 여러 서브 칩을 사용하여 복잡한 검증을 수행합니다:

1. MpiChip (Multiple Precision Integer)

목적: 큰 정수를 여러 16비트 limb로 분해하여 표현

사용처:

- `rw_counter` : 2 limbs (32비트)
- `id` : 2 limbs (32비트)
- `address` : 10 limbs (160비트)

구현:

```
pub struct Config<T, const N: usize> {
pub limbs: [Column<Advice>; N],
}
```

제약 조건:

- 각 limb가 u16 범위에 있음 (룩업)
- limb들로부터 재구성한 값이 원래 값과 일치:

```
value = limbs[0] + limbs[1] * 2^16 + limbs[2] * 2^32 + ...
```

2. RlcChip (Random Linear Combination)

목적: 256비트 storage_key를 RLC로 인코딩

사용처:

- `storage_key` : 32 bytes (256비트)

구현:

```
pub struct Config<const N: usize> {
pub bytes: [Column<Advice>; N], // Little endian
}
```

제약 조건:

- 각 byte가 u8 범위에 있음 (룩업)
- RLC 인코딩이 올바름:

```
encoded = bytes[0] + bytes[1] * r + bytes[2] * r^2 + ... + bytes[31] * r^31
```

여기서 `r` 은 challenge (randomness)

3. BinaryNumberChip

목적: 작은 정수를 이진수로 표현하고 특정 값과의 일치 검증

사용처:

- `tag` : 4비트로 RwTableTag 표현
- `first_different_limb` : 5비트로 LimbIndex 표현

제약 조건:

- 각 비트가 0 또는 1
- 비트들로부터 재구성한 값이 원래 값과 일치

4. BatchedIsZeroChip

목적: 여러 값이 모두 0인지 효율적으로 검증

사용처:

- `is_non_exist` : AccountStorage와 Account에서 비존재 증명 판단
- `field_tag - CodeHash`
- `initial_value` (committed value)
- `value` (new value)

동작: 세 값이 모두 0이면 `is_non_exist = 1` , 아니면 `is_non_exist = 0`

5. LookupsChip

목적: 범위 검증을 위한 고정 테이블 제공

제공 테이블:

- `u8` : 0~255 (8비트 범위)
- `u10` : 0~1023 (10비트 범위, 스택 주소용)
- `u16` : 0~65535 (16비트 범위, limb용)
- `call_context_field_tag` : CallContextFieldTag 열거형 값들

State Circuit 설정 (StateCircuitConfig)

```
pub  struct StateCircuitConfig<F> {

selector: Column<Fixed>,

rw_table: RwTable,

sort_keys: SortKeysConfig,

initial_value: Column<Advice>, // 초기값 (MPT committed value)
```

```
is_non_exist: BatchedIsZeroConfig, // 비존재 여부

mpt_proof_type: Column<Advice>, // MPT 증명 타입

state_root: Column<Advice>, // 상태 루트

lexicographic_ordering: LexicographicOrderingConfig,

not_first_access: Column<Advice>, // 첫 접근이 아님

lookups: LookupsConfig,

power_of_randomness: [Expression<F>; 31], // r, r^2, ..., r^31

mpt_table: MptTable,

}
```

주요 컬럼 설명

initial_value:

- 각 키 그룹의 첫 접근 시 초기값
- Account/AccountStorage: MPT의 committed value
- 기타: 0

is_non_exist:

- AccountStorage: `initial_value == 0 && value == 0 && field_tag == CodeHash` 이면 비존재
- Account: `initial_value == 0 && value == 0 && field_tag == CodeHash` 이면 비존재

mpt_proof_type:

- AccountStorage:
- 비존재: `MPTProofType::StorageDoesNotExist`
- 존재: `MPTProofType::StorageChanged`

- Account:
- 비존재: `MPTProofType::AccountDoesNotExist`
- 존재: `field_tag` 값 (Nonce, Balance, CodeHash 등)

state_root:

- 각 행에서의 상태 루트
- 마지막 접근 후 MPT 업데이트가 있으면 변경됨
- offset-1에 할당됨 (상태 루트는 마지막 접근 행에서 변경)

not_first_access:

- 같은 키 그룹 내에서 첫 접근이 아니면 1
- `first_different_limb` 이 RwCounter0 또는 RwCounter1이면 1

제약 조건 (Constraints)

State Circuit의 제약 조건은 `ConstraintBuilder` 를 통해 구축됩니다.

일반 제약 조건

1. is_write는 boolean:

```
is_write * (1 - is_write) = 0
```

2. not_first_access 정의:

```
not_first_access = first_different_limb[0] * first_different_limb[1]
* first_different_limb[2] * first_different_limb[3]
```

처음 4개 limb 비트가 모두 1이면 (즉, first_different_limb >= 16) not_first_access = 1

3. 첫 접근 시 읽기는 값을 변경하지 않음:

```
first_access * is_read * (value - initial_value) = 0
```

4. 첫 접근 시 value_prev는 initial_value:

```
first_access → value_prev_column = initial_value
```

5. 첫 접근이 아닐 때 읽기는 값을 변경하지 않음:

```
not_first_access * is_read * (value - value_prev) = 0
```

6. 같은 키 그룹 내에서 initial_value는 불변:

```
not_first_access → initial_value = initial_value_prev
```

RW 타입별 제약 조건

Start (패딩)

```
fn build_start_constraints(&mut self, q: &Queries<F>) {
// 사용하지 않는 키는 0
self.require_zero("field_tag is 0", q.field_tag());
self.require_zero("address is 0", q.rw_table.address);
self.require_zero("id is 0", q.id());
self.require_zero("storage_key is 0", q.rw_table.storage_key);
// rw_counter는 매 행마다 1씩 증가
```

```
self.require_zero(
"rw_counter increases by 1",
q.lexicographic_ordering_selector * (q.rw_counter_change() - 1)
);
// 값과 초기값은 0
self.require_zero("value is 0", q.value());
self.require_zero("initial_value is 0", q.initial_value());
// state_root는 불변
self.require_equal("state_root unchanged", q.state_root(), q.state_root_prev());
}
```

Memory

```
fn build_memory_constraints(&mut self, q: &Queries<F>) {
// 사용하지 않는 키는 0
self.require_zero("field_tag is 0", q.field_tag());
self.require_zero("storage_key is 0", q.rw_table.storage_key);
// 첫 접근 시 읽기는 0
self.require_zero(
"first read is 0",
q.first_access() * q.is_read() * q.value()
);
// 메모리 주소는 2 limbs에 맞음 (32비트)
```

```
for limb in &q.address.limbs[2..] {
self.require_zero("address fits into 2 limbs", limb);
}
// 주소는 32바이트 정렬 (limb0 / 32는 u16 범위)
self.add_lookup("address aligned", vec![
(q.address.limbs[0] / 32, q.lookups.u16)
]);
// 초기값은 0
self.require_zero("initial_value is 0", q.initial_value());
// state_root는 불변
self.require_equal("state_root unchanged", q.state_root(), q.state_root_prev());
// value_prev 일관성
self.condition(q.not_first_access(), |cb| {
cb.require_equal("value_prev consistent",
q.rw_table.value_prev, q.value_prev_column());
});
}
```

Stack

```
fn build_stack_constraints(&mut self, q: &Queries<F>) {
// 사용하지 않는 키는 0
self.require_zero("field_tag is 0", q.field_tag());
self.require_zero("storage_key is 0", q.rw_table.storage_key);
```

```
// 첫 접근은 항상 쓰기
self.require_zero(
"first access is write",
q.first_access() * (1 - q.is_write())
);
// 스택 주소는 10비트 (0~1023)
self.add_lookup("stack address in range", vec![
(q.rw_table.address, q.lookups.u10)
]);
// 같은 call_id 내에서 주소는 0 또는 1씩만 증가
self.condition(q.is_tag_and_id_unchanged(), |cb| {
cb.require_boolean("address change is 0 or 1", q.address_change());
});
// 초기값은 0
self.require_zero("initial_value is 0", q.initial_value());
// state_root는 불변
self.require_equal("state_root unchanged", q.state_root(), q.state_root_prev());
}
```

AccountStorage

```
fn build_account_storage_constraints(&mut self, q: &Queries<F>) {
// field_tag는 항상 CodeHash (비존재 판단용)
```

```
self.require_equal(
"field_tag is CodeHash",
q.field_tag(),
AccountFieldTag::CodeHash.expr()
);
// MPT 증명 타입 결정
let  is_non_exist = q.is_non_exist();
self.require_equal(
"mpt_proof_type correct",
q.mpt_proof_type(),
is_non_exist * StorageDoesNotExist.expr()
+ (1 - is_non_exist) * StorageChanged.expr()
);
// 마지막 접근 시 MPT 룩업
self.condition(q.last_access(), |cb| {
cb.add_lookup("mpt_update lookup", vec![
(1, q.mpt_update_table.q_enable),
(q.rw_table.address, q.mpt_update_table.address),
(q.rw_table.storage_key, q.mpt_update_table.storage_key),
(q.mpt_proof_type(), q.mpt_update_table.proof_type),
(q.state_root(), q.mpt_update_table.new_root),
(q.state_root_prev(), q.mpt_update_table.old_root),
(q.value(), q.mpt_update_table.new_value),
```

```
(q.initial_value(), q.mpt_update_table.old_value),
]);
});
// value_prev 일관성
self.condition(q.not_first_access(), |cb| {
cb.require_equal("value_prev consistent",
q.rw_table.value_prev, q.value_prev_column());
});
}
```

Account

```
fn build_account_constraints(&mut self, q: &Queries<F>) {
// 사용하지 않는 키는 0
self.require_zero("id is 0", q.id());
self.require_zero("storage_key is 0", q.rw_table.storage_key);
// MPT 증명 타입 결정
self.require_equal(
"mpt_proof_type correct",
q.mpt_proof_type(),
q.is_non_exist() * AccountDoesNotExist.expr()
+ (1 - q.is_non_exist()) * q.field_tag()
);
// 마지막 접근 시 MPT 룩업
```

```
self.condition(q.last_access(), |cb| {
cb.add_lookup("mpt_update lookup", vec![
(1, q.mpt_update_table.q_enable),
(q.rw_table.address, q.mpt_update_table.address),
(q.rw_table.storage_key, q.mpt_update_table.storage_key),
(q.mpt_proof_type(), q.mpt_update_table.proof_type),
(q.state_root(), q.mpt_update_table.new_root),
(q.state_root_prev(), q.mpt_update_table.old_root),
(q.value(), q.mpt_update_table.new_value),
(q.initial_value(), q.mpt_update_table.old_value),
]);
});
}
```

CallContext

```
fn  build_call_context_constraints(&mut  self, q: &Queries<F>) {
// 사용하지 않는 키는 0
self.require_zero("address is 0", q.rw_table.address);
self.require_zero("storage_key is 0", q.rw_table.storage_key);
// field_tag는 CallContextFieldTag 범위
self.add_lookup("field_tag in range", vec![
(q.field_tag(), q.lookups.call_context_field_tag)
```

```
]);
// 초기값은 0
self.require_zero("initial_value is 0", q.initial_value());
// state_root는 불변
self.require_equal("state_root unchanged", q.state_root(), q.state_root_prev());
// value_prev는 항상 0
self.require_zero("value_prev is 0", q.value_prev_column());
}
```

TxAccessListAccount / TxAccessListAccountStorage

```
fn  build_tx_access_list_account_constraints(&mut  self, q: &Queries<F>) {
// 사용하지 않는 키는 0
self.require_zero("field_tag is 0", q.field_tag());
self.require_zero("storage_key is 0", q.rw_table.storage_key);
// 값은 boolean (warm=1, cold=0)
self.require_boolean("value is boolean", q.value());
// 초기값은 false (cold)
self.require_zero("initial_value is false", q.initial_value());
// state_root는 불변
self.require_equal("state_root unchanged", q.state_root(), q.state_root_prev());
}
```

TxRefund

```
fn build_tx_refund_constraints(&mut self, q: &Queries<F>) {
// 사용하지 않는 키는 0
self.require_zero("address is 0", q.rw_table.address);
self.require_zero("field_tag is 0", q.field_tag());
self.require_zero("storage_key is 0", q.rw_table.storage_key);
// 초기값은 0
self.require_zero("initial_value is 0", q.initial_value());
// state_root는 불변
self.require_equal("state_root unchanged", q.state_root(), q.state_root_prev());
}
```

TxLog

```
fn build_tx_log_constraints(&mut self, q: &Queries<F>) {
// 항상 쓰기
self.require_equal("is_write is true", q.rw_table.is_write, 1.expr());
// 초기값은 0
self.require_zero("initial_value is 0", q.initial_value());
// state_root는 불변
self.require_equal("state_root unchanged", q.state_root(), q.state_root_prev());
```

```
}
```

AccountTransientStorage (EIP-1153)

```
fn build_account_transient_storage_constraints(&mut self, q: &Queries<F>) {
// field_tag는 0
self.require_zero("field_tag is 0", q.field_tag());
// 초기값은 0 (트랜잭션 시작 시 모두 0)
self.require_zero("initial_value is 0", q.initial_value());
// value_prev 일관성
self.condition(q.not_first_access(), |cb| {
cb.require_equal("value_prev consistent",
q.rw_table.value_prev, q.value_prev_column());
});
// state_root는 불변 (임시 스토리지는 MPT에 저장 안 됨)
self.require_equal("state_root unchanged", q.state_root(), q.state_root_prev());
}
```

MPT 증명 타입별 제약 조건

State Circuit은 MPT Circuit과 연동하여 상태 변경을 검증합니다.

MPTProofType 열거형

```
pub enum MPTProofType {
// Account 관련
AccountDoesNotExist = 1, // 계정 비존재
NonceChanged, // Nonce 변경
BalanceChanged, // Balance 변경
CodeHashExists, // Keccak CodeHash 존재
PoseidonCodeHashExists, // Poseidon CodeHash 존재
CodeSizeExists, // CodeSize 존재
// Storage 관련
StorageDoesNotExist, // 스토리지 슬롯 비존재
StorageChanged, // 스토리지 값 변경
}
```

AccountFieldTag와 MPTProofType 매핑

```
impl From<AccountFieldTag> for MPTProofType {
fn from(tag: AccountFieldTag) -> Self {
match tag {
AccountFieldTag::Nonce => Self::NonceChanged,
AccountFieldTag::Balance => Self::BalanceChanged,
AccountFieldTag::KeccakCodeHash => Self::CodeHashExists,
AccountFieldTag::CodeHash => Self::PoseidonCodeHashExists,
AccountFieldTag::CodeSize => Self::CodeSizeExists,
```

```
AccountFieldTag::NonExisting => Self::AccountDoesNotExist,
}
}
}
```

MPT 룩업 조건

Account 마지막 접근:

```
(address, storage_key=0, proof_type, new_root, old_root, new_value, old_value)
```

- `proof_type` : `is_non_exist ? AccountDoesNotExist : field_tag`
- `new_root` : 현재 state_root
- `old_root` : 이전 state_root
- `new_value` : 현재 value
- `old_value` : initial_value (committed value)

AccountStorage 마지막 접근:

```
(address, storage_key, proof_type, new_root, old_root, new_value, old_value)
```

- `proof_type` : `is_non_exist ? StorageDoesNotExist : StorageChanged`
- `new_root` : 현재 state_root
- `old_root` : 이전 state_root
- `new_value` : 현재 value
- `old_value` : initial_value (committed value)

비존재 증명 (Non-Existence Proof)

판단 조건:

```
is_non_exist = (field_tag == CodeHash) && (initial_value == 0) && (value == 0)
```

AccountStorage:

- `is_non_exist = 1` : 스토리지 슬롯이 존재하지 않음 (0→0 전환)
- `is_non_exist = 0` : 스토리지 슬롯이 존재함 (값 변경)

Account:

- `is_non_exist = 1` : 계정이 존재하지 않음 (CodeHash가 0→0)
- `is_non_exist = 0` : 계정이 존재함 (필드 변경)

State Root 관리

State Root 업데이트 시점

State root는 다음 조건에서 업데이트됩니다:

1. **마지막 접근 (last_access):** 같은 키 그룹의 마지막 행
2. **MPT 업데이트 존재:** Account 또는 AccountStorage의 마지막 접근

```
if  is_first_access {
// 이전 행이 마지막 접근이었음
if  let Some(update) = updates.get(prev_row) {
let (new_root, old_root) = update.root_assignments(randomness);
assert_eq!(state_root, old_root);
```

```
state_root = new_root; // 상태 루트 업데이트
}
}
```

State Root 할당

State root는 offset-1에 할당됩니다:

```
// offset != 0일 때
region.assign_advice(
|| "state_root",
self.state_root,
offset - 1, // 이전 행에 할당
|| state_root,
)?;
```

이는 상태 루트가 마지막 접근 행에서 변경되기 때문입니다.

시작 및 종료 State Root

```
pub struct StateCircuitExports<V> {
pub start_state_root: (Cell, Value<V>), // 블록 시작 시 상태 루트
pub end_state_root: (Cell, Value<V>), // 블록 종료 시 상태 루트
}
```

- `start_state_root` : 첫 번째 실제 행의 state_root

- `end_state_root` : 마지막 행의 state_root

이 값들은 Public Input Circuit에서 블록 해시 검증에 사용됩니다.

Witness 할당 프로세스

1. 행 준비 및 패딩

```
let (rows, padding_length) = RwMap::table_assignments_prepad(rows, n_rows);
```

- 실제 RW 연산들을 정렬
- 앞부분에 Start 타입 패딩 추가
- 총 `n_rows` 개 행으로 맞춤

2. 각 행 할당

```
for (offset, row) in  rows.iter().enumerate() {
// 1. Selector 활성화
region.assign_fixed(|| "selector", self.selector, offset, || Value::known(F::one
// 2. Sort keys 할당
tag_chip.assign(region, offset, &row.tag())?;
self.sort_keys.rw_counter.assign(region, offset, row.rw_counter() as u32)?;
self.sort_keys.id.assign(region, offset, row.id().unwrap_or_default() as u32)?;
self.sort_keys.address.assign(region, offset, row.address().unwrap_or_default())
self.sort_keys.storage_key.assign(region, offset, randomness, row.storage_key().
// 3. Lexicographic ordering 검증
```

```
if offset > 0 {
let prev_row = &rows[offset - 1];
let index = self.lexicographic_ordering.assign(region, offset, row, prev_row)?;
let is_first_access = !matches!(index, LimbIndex::RwCounter0 | LimbIndex::RwCou
region.assign_advice(
|| "not_first_access",
self.not_first_access,
offset,
|| Value::known(if is_first_access { F::zero() } else { F::one() }),
)?;
// 4. State root 업데이트
if is_first_access {
if let Some(update) = updates.get(prev_row) {
let (new_root, old_root) = update.root_assignments(randomness);
assert_eq!(state_root, old_root);
state_root = new_root;
}
}
}
// 5. Initial value 할당
let initial_value = randomness.map(|randomness| {
updates.get(row)
.map(|u| u.value_assignments(randomness).1)
```

```
.unwrap_or_default()
});
region.assign_advice(|| "initial_value", self.initial_value, offset, || initial_
// 6. is_non_exist 검증
let  is_non_exist_inputs = randomness.map(|randomness| {
let (_, committed_value) = updates.get(row)
.map(|u| u.value_assignments(randomness))
.unwrap_or_default();
let  value = row.value_assignment(randomness);
[
F::from(row.field_tag().unwrap_or_default()) - F::from(AccountFieldTag::CodeHash
committed_value,
value,
]
});
BatchedIsZeroChip::construct(self.is_non_exist.clone())
.assign(region, offset, is_non_exist_inputs)?;
// 7. MPT proof type 결정
let  mpt_proof_type = is_non_exist_inputs.map(|[_field_tag, committed_value, val
F::from(match  row {
Rw::AccountStorage { .. } => {
if  committed_value.is_zero_vartime() && value.is_zero_vartime() {
MPTProofType::StorageDoesNotExist as u64
```

```
} else {
MPTProofType::StorageChanged as u64
}
}
Rw::Account { field_tag, .. } => {
if  committed_value.is_zero_vartime()
&& value.is_zero_vartime()
&& matches!(field_tag, AccountFieldTag::CodeHash)
{
MPTProofType::AccountDoesNotExist as u64
} else {
*field_tag  as u64
}
}
_ => 0,
})
});
region.assign_advice(|| "mpt_proof_type", self.mpt_proof_type, offset, || mpt_pr
// 8. State root 할당 (offset-1에)
if  offset != 0 {
let  assigned = region.assign_advice(
|| "state_root",
self.state_root,
```

```
offset - 1,
|| state_root,
)?;
if  start_state_root.is_none() {
start_state_root.replace(assigned);
}
}
}
```

3. 마지막 행 처리

```
if  offset + 1 == rows_len {
// 마지막 행도 마지막 접근이므로 MPT 업데이트 확인
if  let Some(update) = updates.get(row) {
state_root = randomness.zip(state_root).map(|(randomness, state_root)| {
let (new_root, old_root) = update.root_assignments(randomness);
if !state_root.is_zero_vartime() {
assert_eq!(state_root, old_root);
}
new_root
});
}
let  assigned = region.assign_advice(
```

```
|| "last row state_root",
self.state_root,
offset,
|| state_root,
)?;
end_state_root.replace(assigned);
}
```

병렬 할당 (Parallel Assignment)

`parallel_syn` 기능이 활성화되면 병렬 할당을 사용할 수 있습니다:

```
#[cfg(feature = "parallel_syn")]
fn assign_par(&self, layouter: &mut impl Layouter<F>, ...) -> Result<(), Error
let num_threads = std::thread::available_parallelism().unwrap().get();
let chunk_size = (rows_len + num_threads - 1) / num_threads;
// Phase 1: RW 테이블 할당 (병렬)
layouter.assign_regions(
|| "state circuit part1",
rows.chunks(chunk_size).map(|rows| {
move |mut region: Region<'_, F>| {
self.rw_table.load_with_region_part(&mut region, rows, randomness)
}
```

```
}).collect_vec(),
)?;
// Phase 2: Sort keys 및 제약 조건 할당 (병렬)
let is_first_access_chunks = layouter.assign_regions(
|| "state circuit part2",
indices.chunks(chunk_size).map(|indices| {
move |mut region: Region<'_, F>| {
self.assign_with_region_part(&mut region, rows, indices, updates, randomness)
}
}).collect_vec(),
)?;
// Phase 3: State root 할당 (순차)
layouter.assign_region(
|| "state circuit part3",
|mut region| {
self.assign_state_roots(
&mut region, &rows, padding_length,
&is_first_access_vec, updates, randomness
)
},
)?;
}
```

병렬 할당은 다음과 같이 제어할 수 있습니다:

```
# 순차 할당
STATE_CIRCUIT_ASSIGNMENT_TYPE=serial


# 병렬 할당 (기본값)
STATE_CIRCUIT_ASSIGNMENT_TYPE=parallel
```

동작 예시

예시 1: 계정 잔액 변경

시나리오: 계정 `0x1234...` 의 잔액을 100에서 200으로 변경

RW 테이블 행:

| rw_counter | is_write | tag | id | address | field_tag | storage_key | value | value_prev | initial_value |

|----------|----------|----|---|---------|----------| ----------|-----|----------|----------|

| 100 | 0 | Account | 1 | 0x1234... | Balance | 0 | 100 | 0 | 100 |

| 101 | 1 | Account | 1 | 0x1234... | Balance | 0 | 200 | 100 | 100 |

검증 과정:

1. **첫 번째 행 (읽기):**

- `first_access = 1` (tag, id, address, field_tag가 처음 등장)
- `is_read * (value - initial_value) = 0` ✓ (100 - 100 = 0)
- `value_prev_column = initial_value` ✓ (0 = 100은 아니지만 첫 접근이므로 initial_value 할당)

2. **두 번째 행 (쓰기):**

- `first_access = 0` (rw_counter만 다름)
- `not_first_access = 1`
- `is_write = 1`
- `value_prev = 100` (이전 행의 value)
- `last_access = 1` (다음 행이 다른 키)

3. **MPT 룩업:**

- `address = 0x1234...`
- `storage_key = 0`
- `proof_type = BalanceChanged`
- `new_root = state_root_after`
- `old_root = state_root_before`
- `new_value = 200`
- `old_value = 100` (initial_value)

4. **State Root 업데이트:**

- 두 번째 행이 마지막 접근이므로 state_root 업데이트
- `state_root: state_root_before → state_root_after`

예시 2: 스토리지 슬롯 생성

시나리오: 계정 `0x5678...` 의 스토리지 슬롯 `0xabcd...` 에 값 50 쓰기 (이전에 존재하지 않음)

RW 테이블 행:

| rw_counter | is_write | tag | id | address | field_tag | storage_key | value | value_prev | initial_value |

|---|---|---|---|---|---|---|---|---|---|

| 200 | 1 | AccountStorage | 2 | 0x5678... | CodeHash | 0xabcd... | 50 | 0 | 0 |

검증 과정:

1. **첫 번째 행 (쓰기):**

- `first_access = 1`
- `is_write = 1`
- `initial_value = 0` (MPT에 없음)
- `value = 50`
- `last_access = 1`

2. **비존재 증명 판단:**

- `field_tag = CodeHash` ✓
- `initial_value = 0` ✓
- `value = 50` ✗
- `is_non_exist = 0` (값이 0이 아니므로 존재함)

3. **MPT 룩업:**

- `proof_type = StorageChanged` (is_non_exist = 0)
- `old_value = 0`
- `new_value = 50`

예시 3: 스택 연산

시나리오: Call ID 3에서 스택에 값 push

RW 테이블 행:

| rw_counter | is_write | tag | id | address | field_tag | storage_key | value | value_prev | initial_value |

|----------|----------|----|---|---------|----------| ----------|-----|----------|----------|

| 300 | 1 | Stack | 3 | 1023 | 0 | 0 | 0x1234... | 0 | 0 |

| 301 | 1 | Stack | 3 | 1022 | 0 | 0 | 0x5678... | 0 | 0 |

검증 과정:

1. **첫 번째 행:**

- `first_access = 1` (새로운 call_id와 address)
- `is_write = 1` ✓ (첫 접근은 항상 쓰기)
- `address = 1023` ✓ (10비트 범위)
- `initial_value = 0` ✓
- `state_root` 불변 ✓

2. **두 번째 행:**

- `first_access = 1` (address가 다름)
- `is_tag_and_id_unchanged = 1` (tag와 id는 같음)
- `address_change = 1023 - 1022 = 1` ✓ (0 또는 1만 허용)
- `is_write = 1` ✓
- `state_root` 불변 ✓

예시 4: 메모리 읽기/쓰기

시나리오: Call ID 5에서 메모리 주소 64에 값 쓰고 읽기

RW 테이블 행:

| rw_counter | is_write | tag | id | address | field_tag | storage_key | value | value_prev | initial_value |

|----------|----------|----|---|---|---|---|---|---|---|

| 400 | 1 | Memory | 5 | 64 | 0 | 0 | 0xaabb... | 0 | 0 |

| 401 | 0 | Memory | 5 | 64 | 0 | 0 | 0xaabb... | 0xaabb... | 0 |

검증 과정:

1. **첫 번째 행 (쓰기):**

- `first_access = 1`
- `is_write = 1`
- `address = 64` ✓ (2 limbs에 맞음, 32바이트 정렬)
- `address.limbs[0] / 32 = 64 / 32 = 2` ✓ (u16 범위)
- `initial_value = 0` ✓
- `state_root` 불변 ✓

2. **두 번째 행 (읽기):**

- `first_access = 0` (rw_counter만 다름)
- `not_first_access = 1`
- `is_read = 1`
- `value = value_prev = 0xaabb...` ✓ (읽기는 값 변경 안 함)
- `initial_value = 0` ✓ (불변)
- `value_prev_column = value_prev` ✓
- `state_root` 불변 ✓

최적화 기법

1. Limb 분해

큰 정수를 16비트 limb로 분해하여 필드 원소 오버플로우 방지:

- 160비트 주소 → 10 limbs
- 256비트 storage_key → 16 limbs (각 2바이트)

- 32비트 rw_counter → 2 limbs

2. RLC (Random Linear Combination)

Storage key를 RLC로 인코딩하여 단일 필드 원소로 표현:

```
storage_key_rlc = bytes[0] + bytes[1]*r + bytes[2]*r^2 + ... + bytes[31]*r^31
```

이를 통해 룩업과 비교 연산을 효율적으로 수행합니다.

3. Binary Number Chip

작은 열거형 값을 비트로 표현하여 특정 값과의 일치를 효율적으로 검증:

- `tag` : 4비트로 12개 RwTableTag 표현
- `first_different_limb` : 5비트로 32개 LimbIndex 표현

4. Batched IsZero

여러 값이 모두 0인지 한 번에 검증하여 제약 조건 수 감소:

```
is_non_exist = (field_tag - CodeHash == 0) && (initial_value == 0) && (value ==
```

5. 조건부 제약 조건

`ConstraintBuilder` 의 `condition` 메서드를 사용하여 특정 조건에서만 제약 조건 적용:

```
self.condition(q.tag_matches(RwTableTag::Memory), |cb| {
cb.build_memory_constraints(q);
```

```
});
```

이를 통해 각 RW 타입별로 다른 제약 조건을 효율적으로 적용합니다.

6. 병렬 할당

`parallel_syn` 기능을 사용하여 witness 할당을 병렬화:

- Phase 1: RW 테이블 할당 (병렬)
- Phase 2: Sort keys 및 제약 조건 할당 (병렬)
- Phase 3: State root 할당 (순차, 의존성 있음)

관련 문서

이전/다음

- **이전**: EVM Circuit - EVM 실행 검증
- **다음**: Bytecode Circuit - 바이트코드 검증

같은 섹션 (핵심 회로)

- EVM Circuit - RW 연산 생성
- Bytecode Circuit - 바이트코드 검증
- Keccak Circuit - 해시 함수
- Tx Circuit - 트랜잭션 검증
- Copy Circuit - 복사 연산

관련 테이블

- RW Table - RW Table 스키마

- MPT Table - MPT 증명 데이터
- 테이블 시스템 - 테이블 개요

관련 시스템

- 데이터 흐름 - 전체 데이터 흐름
- RW 연산 - RW 연산 생성 과정
- MPT Circuit - MPT 증명 검증
- Witness 생성 - Witness 데이터

구현 세부사항

- 제약 조건 구축 - ConstraintBuilder
- Lookup 메커니즘 - 룩업 동작
- RLC 기법 - 데이터 압축
- 병렬화 - 병렬 처리 전략

참고 자료

코드 파일:

- `zkevm-circuits/src/state_circuit.rs`
- `zkevm-circuits/src/state_circuit/constraint_builder.rs`
- `zkevm-circuits/src/state_circuit/lexicographic_ordering.rs`
- `zkevm-circuits/src/state_circuit/multiple_precision_integer.rs`
- `zkevm-circuits/src/state_circuit/random_linear_combination.rs`
- `zkevm-circuits/src/state_circuit/lookups.rs`

- `zkevm-circuits/src/table.rs`

주요 구조체:

- `StateCircuit<F>` : 메인 회로 구조체
- `StateCircuitConfig<F>` : 회로 설정
- `RwTable` : RW 테이블 정의
- `MptTable` : MPT 테이블 정의
- `SortKeysConfig` : 정렬 키 설정
- `ConstraintBuilder<F>` : 제약 조건 빌더